

Notes for "Databasesystems"

Simon Holm

DM576 - Databasesystems

Contents

1	Introduktion	5
2	Notes	5
2.1	Introduction	5
2.1.1	What is a database	5
2.1.2	Why do we use databases	5
2.1.3	Transactions	6
2.1.4	When developing a Database	6
2.2	Hard drives	6
2.2.1	Seperating records	6
2.3	Relations and schemas	6
2.3.1	Table/relation	6
2.3.2	Using schemas	6
2.4	Relational Algebra	7
2.4.1	What is relational algebra	7
2.4.2	Selection	7
2.4.3	Projection	7
2.4.4	Extended projection	7
2.4.5	Cartesian product	8
2.4.6	Theta-join	8
2.4.7	Natural Join	8
2.4.8	Renaming	8
2.4.9	Tree	8
2.4.10	Bags	8
2.4.11	Extended Algebra	9
2.5	SQL	10
2.5.1	Relational schema to SQL examples	10
2.5.2	Datatypes	10
2.5.3	Date and time	10
2.5.4	Keys	10
2.5.5	Altering a relation	11
2.5.6	Subqueries	11
2.5.7	Triggers	11
2.5.8	Functions	12
2.5.9	Procedures	12
2.5.10	Views	12
2.5.11	Virtual View	12
2.5.12	Materialized View	13
2.5.13	Modifying	13
2.5.14	Transactions	13
2.5.15		13
2.6	SQL statements	13
2.6.1	Basic statements	13
2.6.2	Patterns	13
2.6.3	NULL	14
2.6.4	Set notation	14
2.6.5	IN	14
2.6.6	EXISTS	14
2.6.7	ANY	14
2.6.8	ALL	14
2.6.9	DISTINCT	14
2.6.10	ASC or DESC	15
2.6.11	LIMIT X	15
2.6.12	COUNT(x)	15
2.6.13	HAVING	15

2.6.14	GROUP BY	15
2.6.15	OUTERJOIN	15
2.6.16	FOR EACH ROW	15
2.6.17	COMMIT	15
2.6.18	COMMAND ORDER IN SQL	15
2.7	Database Modification	16
2.8	Database Design	16
2.9	Anomalies	16
2.10	NORMAL FORM	16
2.10.1	Third Normal Form/BCNF	17
2.11	Checking for BCNF	17
2.12	Constraints	17
2.12.1	DEFERABLE	18
2.13	Checks	18
2.14	Using SQL in real languages	18
3	Exercises	19
3.1	Exercise 2.1	19
3.2	Exercise 2.2	20
3.3	Exercise 2.3	20
3.4	Exercise 3.1	21
3.5	Exercise 3.2	23
3.6	Exercise 2.4.14	24
3.7	Exercise 2.4.2	25
3.8	Exercise 5.1.1	25
3.9	Exercise 5.1.3	25
3.10	Exercise 5.1.4	25
3.11	Exercise 6.1.3	26
3.12	Exercise 6.2.2	26
3.13	Exercise 6.1.1	27
3.14	Exercise 6.1.2	27
3.15	Exercise 6.1.5	28
3.16	Exercise 6.2.1	28
3.17	Exercise 6.3.2	29
3.18	Exercise in lecture	29
3.19	Exercise 6.4.1	30
3.20	Exercise 6.4.2	30
3.21	Exercise 6.4.6	30
3.22	Exercise 6.4.7	31
3.23	Exercise 6.5.1	31
3.24	Exercise 3.1.1	32
3.25	Exercise 3.2.1	32
3.26	Exercise 3.3.3	32
3.27	Exercise 3.2.10	33
3.28	Exercise 3.3.1	34
3.29	Exercise 4.1.1	34
3.30	Exercise 4.1.2	35
3.31	Exercise 4.1.3	35
3.32	Exercise 4.2.1	35
3.33	Exercise 4.3.1	35
3.34	Exercise 4.4.1	36
3.35	Exercise 4.5.1	37
3.36	Exercise in lecture	37
3.37	Exercise 7.1.1	38
3.38	Exercise 7.1.3	39
3.39	Exercise in lecture	40
3.40	Exercise in lecture	40

3.41	Exercise 7.5.1	41
3.42	Exercise 6.6.1	42
3.43	Exercise 9.4.1	44
3.44	Exercise 9.3.1	44
3.45	Exercise 13.2.1	45
3.46	Exercise 13.3.1	46
3.47	Exercise 13.4.1	46
3.48	Exercise 13.4.2	47
3.49	Exercise 13.4.6	47
3.50	Exercise 13.7.1	47
3.51	Exercise 14.1.1	47
3.52	Exercise 14.2.1	48
3.53	Exercise 14.1.1	48

1 Introduction

Notes and exercises for lectures and TA-sessions.

2 Notes

2.1 Introduction

2.1.1 What is a database

Databases are collections of informations that exists over a long period of time. Therefore, storage of data. We use a DBMS (DataBase Management System) which allows a user to interact with a database. The DBMS is expected to:

- Allow users create a DB
- Enable users to query/modify the DB
- Support storage of large volumes of data (Allow efficient retrieval of data and data modifications)
- Be durable
- Support concurrency
- Be consistent

2.1.2 Why do we use databases

- Easy to use
- Flexible searching
- Efficiency
- Centralized storage, multi-user access
- Scalability (large amounts of data)
- Security and consistency
- Abstraction (implementation hiding)
- Good opportunities for data modeling

2.1.3 Transactions

Transactions are blocks/groups of statements that must be executed together. This is done to prevent issues if a problem occurs during a sequence of statements.

Example: ACID method is required for transactions

- 'A' for "atomicity" – all or nothing of transactions
- 'C' for "consistency" – constraints hold before and after each transaction
- 'I' for "isolation" – illusion of sequential execution of each transaction
- 'D' for "durability" – effect of a completed transaction may not get lost

2.1.4 When developing a Database

- Requirement-specification
- Conceptual modeling (E/R-model for data)
- Logical modeling (Relational data model)
- Query Language (SQL)
- Physical Design (Storage and Indexing)

2.2 Hard drives

- reading time: $\frac{\varphi}{360 \cdot m} \cdot t$
where $m = \text{nonmagnetic\%}$ and $t = \text{time}$
- The I/O Model of Computation
disk access time is much larger than actually performing anything on the data

2.2.1 Separating records

-

Note: a record cannot be stored within 2 blocks. Instead we can split (span) a record to fit. In this case we use *sequencing*, link the blocks.

- When deleting/inserting it is very expensive when a database is sorted in some way. You will have to move the data around

2.3 Relations and schemas

2.3.1 Table/relation

2.3.2 Using schemas

A relational schema gives a relation with a given name some attributes. (optionally: types of attributes)

Example: *Student(student_name, student_id, , student_email,)*

A database is therefore a collection of relations

A database schema is a set of all relation schemas in the database

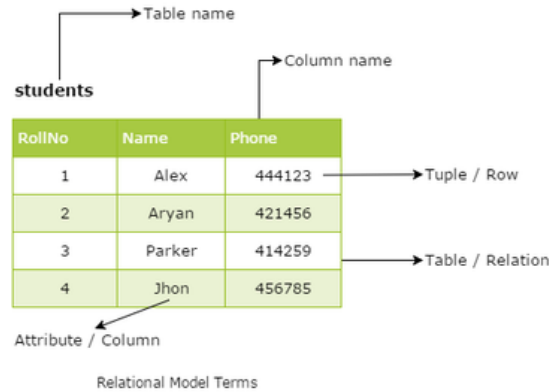


Figure 1: Table/relation

2.4 Relational Algebra

2.4.1 What is relational algebra

- Set operations Much like discrete mathematics, for sets we ignore duplicates
 - Union, Intersections and difference
 - Usual set operations but, *both operand must have the same relation schema*
- Filter operations
 - Selection (σ): picking certain rows
 - Projection (π): picking certain columns
- Products and join: compositions of relations
- Renaming of relations and attributes

2.4.2 Selection

- $R_1 = \sigma_C(R_2)$
 - C is a condition (as in "if" statements), that refers to attributes of R_2
 - R_1 is all the tuples of R_2 that satisfy C

2.4.3 Projection

Projection

- $R_1 = \pi_L(R_2)$
 - L is a list of attributes from the schema of R_2
 - R_1 is constructed by looking at each tuple of R_2 , extracting the attributes on list L , in the order specified, and creating from those components a tuple for R_1
 - Eliminate duplicate tuples, if any

2.4.4 Extended projection

- Using the same π_L operator, we allow the list L to contain arbitrary expressions involving attributes:
 - Arithmetic on attributes, e.g., $A + B \Rightarrow C$
 - Duplicate occurrences of the same attribute

2.4.5 Cartesian product

- $R_3 = R_1 \times R_2$
 - Pair each tuple t_1 of R_1 with each tuple t_2 of R_2
 - Concatenation $t_1 t_2$ is a tuple of R_3
 - Schema of R_3 is the attributes of R_1 and then R_2 , in order
 - But beware attribute A of the same name in R_1 and R_2 : use $R_1.A$ and $R_2.A$

2.4.6 Theta-join

- $R_3 = R_1 \bowtie_C R_2$
 - Take the Cartesian Product $R_1 \times R_2$
 - Then apply σ_C to the result
- As for σ , C can be any boolean-valued condition
 - Historic versions of this operator allowed only $A\theta B$, where θ is $=, <, >$, etc.; hence the name “theta-join”

2.4.7 Natural Join

- $R_3 = R_1 \bowtie R_2$
 - Equating attributes of the same name, and.
 - Projecting out one copy of each pair of equated attributes.
- As for σ , C can be any boolean-valued condition
 - Historic versions of this operator allowed only $A\theta B$, where θ is $=, <, >$, etc.; hence the name “theta-join”

$$\sigma_{\text{launched}>1921 \text{ AND } \text{displacement}>35000} (Classes \bowtie Ships)$$

$$\pi_{\text{name, displacement, numGuns}} (\sigma_{\text{battle=Guadalcanal}} ((Outcomes \bowtie_{\text{ship=name}} Ships) \bowtie Classes))$$

2.4.8 Renaming

- The ρ operator gives a new schema to a relation
- $R_1 := \rho_C(A_1 \dots A_n)(R_2)$ makes R_1 be a relation with attributes $A_1 \dots A_n$ and the same tuples as R_2
- Simplified notation: $R - 1((A_1 \dots A_n)) := R_2$

2.4.9 Tree

2.4.10 Bags

A bag is a set that allows duplicates

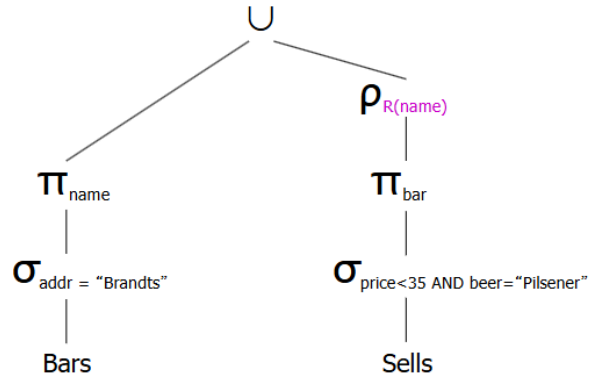


Figure 2: Tree showing the set operators of relational algebra

2.4.11 Extended Algebra

- δ : Eliminate duplicates from bags
 $R_1 = \delta(R_2)$. R_1 is a copy of R_2 with no duplicates
- τ : Sort tuple
ORDER BY
- Aggregation
SUM(), COUNT(), MAX(), MIN(), AVG()
- γ : Grouping and aggregation
 $\gamma_{L,AGG}(R)$

Example: R)

A	B	C
1	2	3
4	5	6
1	2	5

$$\gamma_{A,B,AVG(C) \rightarrow X}(R) =$$

2.5 SQL

2.5.1 Relational schema to SQL examples

```
Course(CourseID, Name, Semester)
CREATE TABLE Course (
    CourseID    CHAR(5)    PRIMARY KEY,
    Name        VARCHAR(20),
    Semester    INT,
);
```

```
Student(StudentID, Name, Address)
CREATE TABLE Student (
    StudentID   INT,        PRIMARY KEY,
    Name        CHAR(20),
    Semester    INT,
);
```

```
StudentCourse(StudentID, CourseID, Year)
CREATE TABLE StudentCourse (
    StudentID   INT,
    CourseID    CHAR(20),
    Year        DATE,
    PRIMARY KEY(StudentID,CourseID,Year)
);
```

Note: use [if exist] to check if table exists before accessing said table.

2.5.2 Datatypes

- INT/INTERGER = a whole number
- FLOAT = decimal number
- CHAR(n) = a string with a fixed n length (use when possible).
- VARCHAR(n) = a string of length $0 \dots n$
- DATE = date

2.5.3 Date and time

- DATE '2021-02-01' for 1st of february 2021.
- TIME '10:48:02,5' for two and a half seconds past 10:48

2.5.4 Keys

When ever a variable is unique. If in doubt, try identifying some piece of data only using its primary keys. Both the **UNIQUE** and **PRIMARY KEY** constraints provide a guarantee for uniqueness for a column or set of columns.

A **PRIMARY KEY** constraint automatically has a **UNIQUE** constraint.

However, you can have many **UNIQUE** constraints per table, but only one **PRIMARY KEY** constraint per table. If more than one variable points to the **PRIMARY KEY** do as shown in (Relational schema to SQL examples: StudentCourse)

- **UNIQUE**
- **PRIMARY KEY**

2.5.5 Altering a relation

- To delete an attribute:

ALTER TABLE <name>DROP <attribute>;

- To add an attribute:

ALTER TABLE <name>ADD <attribute>;

2.5.6 Subqueries

- A query that is apart of another query
- Can appear from clauses
- Can return a single constant which can be compared to another value in a **WHERE** clause.
- can return relations, which can be compared to another value in **WHERE** clauses.
- If a subquery is guaranteed to produce one tuple, then the subquery can be used as a value

2.5.7 Triggers

Row vs Statement level triggers

- When to use row-level Triggers
 - Auditing individual row changes
 - Enforcing data validation and business rules.
 - automatically updating related tables
 - generating derived or default values
- When to use statement-level triggers
 - Bulk operations
 - auditing or restricting bulk operations
 - enforcing business rules for batch operations
 - Performance considerations
 - When you don't need a trigger to run for each row, improving efficiency
 - DDL Monitoring
 - When tracking schema modifications
 - Aggregation Updates
 - When recalculating summary tables after batch operations.
- The event
 - event execute SQL statements (like **INSERT**, **UPDATE**, **DELETE**, etc.) at predefined time
 - Trigger type like
 - Common use cases:
 - **BEFORE**: Modify data, enforce constraints
 - **AFTER**: Logging, audit trails, updating related tables
 - **INSTEAD OF**: Managing updates/deletes on views

2.5.8 Functions

SYNTAX FOR FUNCTIONS IN SQL

```
CREATE FUNCTION CheckRipoff() RETURNS TRIGGER AS
$$
BEGIN
    IF NEW.price > OLD.price+10 THEN
        INSERT INTO RipoffBars VALUES (NEW.bar);
    END IF;
    RETURN NEW;
END
$$
LANGUAGE plpgsql;
```

- **OR REPLACE** Overrides a function
- function can use many return types like **VOID**, (Traditional data types like **INT**, **VARCHAR**, ect...), Composite Types (e.g. **ROW** and **RECORD**), **SETOF** (e.g **ROWS**, **RECORDS**), **TABLE**, **TRIGGER** Etc.

2.5.9 Procedures

Similar to function, but no return type.

Also space in between **BEGIN** and **END** is more loose (e.g. you can commit)

```
CREATE PROCEDURE name(<arguments>) AS
$$
BEGIN
    PROGRAM
END
$$
LANGUAGE plpgsql;
```

Procedures uses parameters

- **IN** function uses value, does not change
- **OUT** function changes, does not use
- **INOUT** both

2.5.10 Views

A view is a relation defined in terms of stored tables (*base tables*) and other views:

Two kinds of views

- **Virtual:** not stored in the database; just a query for constructing the relation
- **Materialized:** actually constructed and stored

```
CREATE [MATERIALIZED] VIEW <name> AS
<query>
```

2.5.11 Virtual View

- For a view default is virtual.
- A materialized view will always return the latest data to you, however it is not good in performance.
- only a query is stored
- no need to change the view when the base table changes
- expensive when accessing the view often

2.5.12 Materialized View

- Each time a base table changes, the materialized view may change, this costs a lot to recompute
- Save the results of the underlying query in a table (this will improve performance, but the returned data is only as up to date as the last time the materialized view has been refreshed.)

2.5.13 Modifying

- Generally, it is impossible to modify a virtual view, because it does not exist
- SQL provides some rules for when modifications to a view are permitted.
- Roughly, they permit modifications on views that are defined by selecting (using SELECT, not SELECT DISTINCT) some attributes from one relation R
- Rules
 - The WHERE clause must not involve R in a subquery
 - The FROM clause can only consist of one occurrence of R and no other relation.
 - The list in the SELECT clause must include enough attributes that for every tuple inserted into the view. (For example, it is not permitted to project out an attribute that is declared NOT NULL and has no default.)
- Alternatively use INSTEAD OF

2.5.14 Transactions

2.5.15

2.6 SQL statements

2.6.1 Basic statements

- SELECT: selects desired attributes
- FROM: one or more tables
- WHERE: condition about tuples of the tables
 - Boolean operators AND, OR or NOT
 - Comparison operators =, <>(≠), <, >, <= or >=

```
SELECT drinker, beer,  
       price * 0.134 AS priceInEuro  
FROM Sells;  
WHERE beer = Odense Classic
```

- AS: rename an attribute

2.6.2 Patterns

- A condition can compare a string to a pattern by;
 - <Attribute> LIKE <Pattern> or
 <Attribute> NOT LIKE <Pattern>
- Pattern is a quoted string with'
 - % = "any string"
 - _ = "any character"

```

SELECT name
FROM people;
WHERE CPR LIKE '161202 _ _ _ _'

```

2.6.3 NULL

- The logic of conditions in SQL is really 3-valued logic: TRUE, FALSE, UNKNOWN
- Comparing any value (including NULL itself) with NULL yields UNKNOWN
- A tuple is in a query answer iff the WHERE clause is TRUE (not FALSE or UNKNOWN)
 - TRUE = 1; FALSE = 0; UNKNOWN = 1/2
 - AND = MIN; OR = MAX; NOT(x) = 1-x

2.6.4 Set notation

- R UNION S
- R INTERSECTION S
- R DIFFERENCE S

2.6.5 IN

- <tuple> IN (<subquery>) is true if and only if the tuple is a member of the relation produced by the subquery
 - ”check if a tuple exists in the subquery”
- Opposite: <tuple> NOT IN (<subquery>)

2.6.6 EXISTS

- EXISTS(<subquery>) is true if and only if the subquery result is not empty
 - check if a subquery is not empty
- Opposite: NOT EXISTS(<subquery>)

2.6.7 ANY

- x = ANY(<subquery>) is a boolean condition that is true iff x equals at least one tuple in the subquery result
 - check if item x is equal to at least one subquery result.
- the use of x >=<=<> ANY(<subquery>) is allowed

2.6.8 ALL

- x = ALL(<subquery>) is true iff for every tuple t in the relation, x is equal to t
 - check if x is equal to all tuples
- the use of x >=<=<> ALL(<subquery>) is allowed
 - Note:** for x <> ALL(<subquery>) means that x is not present in the subquery as a result

2.6.9 DISTINCT

Treats SELECT DISTINCT as a set with (no duplicates)

2.6.10 ASC or DESC

Ascenting(default) or Descending

2.6.11 LIMIT X

Limit select to the first

```
SELECT A, B
FROM R
WHERE B > 10
ORDER BY A ASC/DESC
LIMIT 1
```

2.6.12 COUNT(x)

Count the number of tuples

2.6.13 HAVING

2.6.14 GROUP BY

2.6.15 OUTERJOIN

LEFT OUTER JOIN $\bowtie$, $\bowtie$, $\bowtie$

2.6.16 FOR EACH ROW

123

2.6.17 COMMIT

Commits a transaction in the database, this is now a permanent change.

2.6.18 COMMAND ORDER IN SQL

5. SELECT
1. FROM
2. WHERE
3. GROUP BY
4. HAVING
6. ORDER BY

<i>title</i>	<i>year</i>	<i>length</i>	<i>genre</i>	<i>studioName</i>	<i>starName</i>
Star Wars	1977	124	SciFi	Fox	Carrie Fisher
Star Wars	1977	124	SciFi	Fox	Mark Hamill
Star Wars	1977	124	SciFi	Fox	Harrison Ford
Gone With the Wind	1939	231	drama	MGM	Vivien Leigh
Wayne's World	1992	95	comedy	Paramount	Dana Carvey
Wayne's World	1992	95	comedy	Paramount	Mike Meyers

$\text{title year} \rightarrow \text{length genre studioName}$ **TRUE**
 $\text{title year} \rightarrow \text{starName}$ **FALSE**

Figure 3: Functional dependency on movie relation

2.7 Database Modification

- Insertion

```
INSERT INTO <relation>
VALUES (<list of values>)
```

If an attribute is not defined, it will be NULL by default (EXCEPT FOR PRIMARY KEYS)

- Deletion

```
DELETE FROM <relation>
WHERE <condition>
```

2.8 Database Design

Functional dependencies (design constraints), sometimes called FD's:

- $X \rightarrow Y$ If two tuples, or more, of relation R agree on all attributes of X they must also agree on all attributes of Y .

We know that $A \rightarrow \{B, C\} = A \rightarrow B \wedge A \rightarrow C$

- Keys of relation
 - K is a *superkey* for relation R if K functionally determines all of R ***
if there ANY of A, B , OR C is redundant in $\{A, B, C\} \rightarrow \{D\}$
 - K is a *key* for R if K is a superkey but no proper subset of K is a superkey.***
no redundant attributes in for A in $A \rightarrow B$

2.9 Anomalies

- Update anomaly: if an update is made, the update needs to happen in all places.
- Deletion anomaly: if an attribute is uninformed, we shall not lose data.

2.10 NORMAL FORM

1. NF The key
2. NF The whole key
3. NF/BCNF And nothing but the key
... so help me Codd

Note: A relation is in BCNF whenever $X \rightarrow Y$ is a nontrivial FD where X a superkey. (Where nontrivial mean Y is not contained in X)

Properties of decomposition

1. Elimination of Anomalies

- Redundancy
- Update Anomalies
- Deletion Anomalies

- Recoverability of Information
- Preservation of Dependencies

Note: BCNF provides (1) and (2) but not necessarily (3).

2.10.1 Third Normal Form/BCNF

- $X \rightarrow A$ violates 3NF *iff* X is not a superkey, and also A is not prime
- Or: $X \rightarrow A$ holds 3NF *iff* X is a superkey or A is prime (i.e. member of any key)

2.11 Checking for BCNF

1. Check if all FD's are superkeys (e.g. for all FD's on relation $R(X, Y, Z)$, all FD points to XYZ)
2. If no violation return R, if there is a violations use the following decomposing rules.
 - 1.

Note: decompose until all has length 2.

2.12 Constraints

Relationship among data that DBMS must enforce

- Keys
- Foreign-keys, or referential-integrity
- Value-based constraints
 - Constrains values of a particular attribute
-

```
CREATE TABLE beers (  
  name    CHAR(20)    PRIMARY KEY,  
  manf    CHAR(20) );  
  
CREATE TABLE Sells (  
  bar     CHAR(20),  
  beer    CHAR(20),  
  price   REAL  
  FOREIGN KEY(beer) REFERENCES Beers(name)  
);
```

2.12.1 DEFERABLE

- NOT DEFERABLE

For each row, constraints are checked

- DEFERABLE INITIALLY IMMEDIATE

- DEFERABLE INITIALLY DEFERED

Check constraints at the end of the transaction

- RESTRICT

- CASCADE

- SET NULL

2.13 Checks

Attribute Check

```
CREATE TABLE Sells (  
  bar      CHAR(20),  
  beer     CHAR(20)    CHECK (beer IN (SELECT name FROM Beers)),  
  price    INT         CHECK (price <= 100)  
);
```

The example almost imitates the use of foreign keys, but is not updated. So no good :(. This is because checks only happen when the attribute itself is checked.

2.14 Using SQL in real languages

1. Code in a specialized language stored in the database itself (e.g. PSM or PL/pgsql)
2. SQL statements are embedded in a *host language* (e.g. Java)
3. Connection tools are used to allow a conventional language to access database

C CLI

Java JDBC

```
import java.sql.*;  
...  
Class.forName("org.postgresql.Driver");  
Connection myCon=DriverManager.getConnection(<URL>);...
```

Python psycopg2

```
import psycopg2;  
  
con_str="host='LocalHost' "
```

3 Exercises

For exercises some exercises may not be numbered due to being only shown in lectures.

3.1 Exercise 2.1

In Fig. 1 and 2 are instances of two relations that might constitute part of a banking database. Indicate the following:

<i>acctNo</i>	<i>type</i>	<i>balance</i>
12345	savings	12000
23456	checking	1000
34567	savings	25

Figure 4: The relation 'Accounts'

<i>firstName</i>	<i>lastName</i>	<i>idNo</i>	<i>account</i>
Robbie	Banks	901-222	12345
Lena	Hand	805-333	12345
Lena	Hand	805-333	23456

Figure 5: The relation Customers

- a The attributes of each relation.

accNo, type, balance

firstName, lastName, idNo, account

- b The tuples of each relation.

Relation **Accounts**:

(12345, savings, 12000)

(23456, checking, 1000)

(34567, savings, 25)

Relation **Customers**:

(Robbie, Banks, 901-222, 12345)

(Lena, Hand, 805-333, 12345)

(Lena, Hand, 805-333, 23456)

- c The components of one tuple from each relation.

Relation **Accounts**, **tuple 1**:

12345, savings, 12000

Relation **Customers**, **tuple 3**:

Lena, Hand, 805-333, 23456

- d The relation schema for each relation.

Accounts(accNo, type, balance)

Customers(firstName, lastName, idNo, account)

- e The database schema.

(Accounts(accNo, type, balance), Customers(firstName, lastName, idNo, account))

f A suitable domain for each attribute.

Relation **Accounts**: **accNo**:

INT

Relation **Accounts**: **type**:

VARCHAR(10)

Relation **Accounts**: **balance**:

DOUBLE

Relation **Customers**, **firstName**:

VARCHAR(20)

Relation **Customers**, **lastName**:

VARCHAR(20)

Relation **Customers**, **idNo**:

VARCHAR(7)

Relation **Customers**, **account**:

INT

g Another equivalent way to present each relation.

Relation **Accounts**:

Account number 12345 has 12000 in their savings account

Account number 23456 has 1000 in their checking account

Account number 34567 has 25 in their savings account

Relation **Customer**:

Robbie Banks with id-Number 901-222 uses account 12345

Lena Hand with id-Number 805-333 uses account 12345

Lena Hand with id-Number 805-333 uses account: 23456

3.2 Exercise 2.2

In Section 2.7 (*the book*) we suggested that there are many examples of attributes that are created for the purpose of serving as keys of relations. Givesome additional examples.

OrderNo

StudentID

3.3 Exercise 2.3

How many different ways (considering orders of tuples and attributes) are there to represent a relation instance if that instance has:

- Three attributes and three tuples, like the relation Accounts of Fig. 6?

$3! = 6$ different ways

- Four attributes and five tuples?

$4! = 24$ different ways

- n attributes and m tuples?

$n!$ different ways

3.4 Exercise 3.1

In this exercise we introduce one of our running examples of a relational database schema. The database schema consists of four relations, whose schemas are:

```
Product(maker, model, type)
PC(model, speed, ram, hd, price)
Laptop(model, speed, ram, hd, screen, price)
Printer(model, color, type, price)
```

The Product relation gives the manufacturer, model number and type (PC, laptop, or printer) of various products. We assume for convenience that model numbers are unique over all manufacturers and product types; that assumption is not realistic, and a real database would include a code for the manufacturer as part of the model number. The PC-relation gives for each model number that is a PC the speed (of the processor, in gigahertz), the amount of RAM (in megabytes), the size of the hard disk (in gigabytes), and the price. The Laptop-relation is similar, except that the screen size (in inches) is also included. The Printer-relation records for each printer model whether the printer produces color output (true, if so), the process type (laser or ink-jet, typically), and the price.

Write the following declarations:

- a A suitable schema for relation Product.

```
Product(maker, model, type)

CREATE TABLE Product (
    maker      VARCHAR(1)      PRIMARY KEY,
    model      INT,
    type       CHAR(20),
);
```

- b A suitable schema for relation PC.

```
PC(model, speed, ram, hd, price)

CREATE TABLE Product (
    model      INT      PRIMARY KEY,
    speed      FLOAT,
    ram        INT,
    hd         INT,
    price      INT
);
```

- c A suitable schema for relation Laptop.

```
Laptop(model, speed, ram, hd, screen, price)

CREATE TABLE Product (
    model      INT,      PRIVATE KEY
    speed      FLOAT,
    ram        INT,
    hd         INT,
    screen     FLOAT,
    price      INT,
);
```

d A suitable schema for relation Printer.

```
Printer(model, colour, type, price)

CREATE TABLE Product (
    model      INT,          PRIVATE KEY
    colour     BOOL,
    type       CHAR(20),
    price      INT
);
```

e An alteration to your Printerschema from (d) to delete the attribute color.

```
Product(maker, model, type)

CREATE TABLE Product (
    maker      VARCHAR(1),
    model      INT,          PRIVATE KEY
    type       CHAR(20),
);
```

f An alteration to your Laptop schema from (c) to add the attribute od(optical-disk type, e.g., cd or dvd). Let the default value for this attribute be 'none' if the laptop does not have an optical disk.

```
Product(maker, model, type)

CREATE TABLE Product (
    maker      CHAR(1),
    model      INT          PRIMARY KEY,
    type       VARCHAR(20),
);
```

3.5 Exercise 3.2

This exercise introduces another running example, concerning World War II capital ships. It involves the following relations:

```
Classes(class, type, country, numGuns, bore, displacement)
Ships(name, class, launched)
Battles(name, date) Outcomes(ship, battle, result)
```

Ships are built in “classes” from the same design, and the class is usually named for the first ship of that class. The relation *Classes* records the name of the class, the type (‘bb’ for battleship or ‘bc’ for battlecruiser), the country that built the ship, the number of main guns, the bore (diameter of the gun barrel, in inches) of the main guns, and the displacement (weight, in tons). Relation *Ships* records the name of the ship, the name of its class, and the year in which the ship was launched. Relation *Battles* gives the name and date of battles involving these ships, and relation *Outcomes* gives the result (sunk, damaged, or ok) for each ship in each battle. Write the following declarations:

- a A suitable schema for relation *Classes*.

```
Classes(class, type, country, numguns, bore, displacement)

CREATE TABLE Classes (
    class          VARCHAR(20),
    type           CHAR(2),
    country        VARCHAR(20),
    numguns        INT
    bore           INT
    displacement   INT
);
```

- b A suitable schema for relation *Ships*.

```
Ships(name, class, launched)

CREATE TABLE Ships (
    name           VARCHAR(20),
    class          CHAR(2),
    launched       YEAR,
);
```

- c A suitable schema for relation *Battles*.

```
Battles(name, class)

CREATE TABLE Battles (
    name          VARCHAR(20),
    date          DATE,
);
```

- d A suitable schema for relation *Outcomes*.

Outcomes(name, class, launched)

```
CREATE TABLE Outcomes(  
  ship      VARCHAR(20),  
  battle    VARCHAR(20),  
  result    VARCHAR(10)  
);
```

- e An alteration to your Classes-relation from (a) to delete the attribute bore.

Classes(name, class, launched)

```
CREATE TABLE Classes (  
  class      CHAR(20),  
  type       VARCHAR(2),  
  country    CHAR(20),  
  numguns    INT  
  displacement INT  
);
```

- f An alteration to your Ships-relation from (b) to include the attribute yardgiving, the shipyard where the ship was built.

Ships(name, class, launched)

```
CREATE TABLE Ships (  
  name       CHAR(20),  
  class      VARCHAR(2),  
  launched   YEAR,  
  yardgiving VARCHAR(20)  
);
```

3.6 Exercise 2.4.14

This exercise builds upon the products schema of Exercise 3.1. Recall that the database schema consists of four relations, whose schemas are:

```
Product(maker, model, type)  
PC(model, speed, ram, hd, price)  
Laptop(model, speed, ram, hd, screen, price)  
Printer(model, color, type, price)
```

Some sample data for the relation Product is shown in Fig. 20. Sample data for the other three relations is shown in Fig. 21. Manufacturers and model numbers have been “sanitized,” but the data is typical of products on sale at the beginning of 2007.

Write expressions of relational algebra to answer the following queries. You may use the linear notation of Section 4.13 if you wish. For the data of Figs. 20 and 21, show the result of your query. However, your answer should work for arbitrary data, not just the data of these figures.

- a) What PC models have a speed of at least 3.00?

$$\pi_{speed \geq 3.00}(pc)$$

- b) Which manufacturers make laptops with a hard disk of at least 100GB?

$$\pi_{maker}(\sigma_{hd \geq 100}(Product \bowtie Laptop))$$

- c) Find the model number and price of all products (of any type) made by manufacturer B.

$$\pi_{model,price}(\rho_{maker=B}(Product \bowtie (Printer \bowtie (PC \bowtie Laptop))))$$

- d) Find the model numbers of all color laser printers.

$$\pi_{model}(\rho_{color=true}(Printer))$$

- e) Find those manufacturers that sell Laptops, but not PC's.

$$\pi_{maker}((Product \bowtie Laptop) - (Product \bowtie PC))$$

$$\pi_{maker}(\rho_{type=Laptop}(Product) - \rho_{type=PC}(Product))$$

- !f) Find those hard-disk sizes that occur in two or more PC's.

no

3.7 Exercise 2.4.2

3.8 Exercise 5.1.1

- Let PC be the relation of Fig. 2.21(a), and suppose we compute the projection $\pi_{speed}(PC)$ speed(PC). What is the value of this expression as a set? As a bag? What is the average value of tuples in this projection, when treated as a set? As a bag?

Values as a set: {(2.66), (2.10, (1.42), (2.80), (3.20), (2.20), (2.00), (1.86), (2.80), (3.06))}

Average speed for set: 2.3666

Values as a bag: {(2.66), (2.10, (1.42), (2.80), (3.20), (3.20), (2.20), (2.20), (2.00), (2.80), (1.86), (2.80), (3.06))}

Average speed for bag: 2.484

3.9 Exercise 5.1.3

This exercise refers to the “battleship” relations of Exercise 2.4.3.

- The expression $\pi_{bore}(Classes)$ yields a single-column relation with the bores of the various classes. For the data of Exercise 2.4.3, what is this relation as a set? As a bag?

Expressed as a set: (15, 14, 16, 18)

Expressed as a bag: (15, 16, 14, 16, 15, 15, 14, 18)

- Write an expression of relational algebra to give the bores of the ships (not the classes). Your expression must make sense for bags; that is, the number of times a value b appears must be the number of ships that have bore b.

$$\pi_{bore}(Ships \bowtie Classes)$$

3.10 Exercise 5.1.4

- a) The associative law for union: $(R \cup S) \cup T = R \cup (S \cup T)$.

Union definition

3.11 Exercise 6.1.3

- a) Find the model number, speed, and hard-disk size for all PC's whose price is under \$1000

```
SELECT model , speed, hd
FROM PC
WHERE price < 1000
```

- d) Find the model number, memory size, and screen size for all laptops costing more than \$1500

```
SELECT model, ram, screen
FROM laptops
WHERE price > 1500
```

- f) Find the model number and hard-disk size for those PC's that have a speed of 3.2 and a price less than \$2000

```
SELECT model, hd
FROM PC
WHERE speed = 3.2 AND price < 2000
```

3.12 Exercise 6.2.2

- a) Give the manufacturer and speed of laptops with a hard disk at least 30 GB

```
SELECT maker, speed
FROM Laptop L NATURAL JOIN Product P ON L.model = P.model
WHERE L.hd < 30
```

- b) Find the model number and price of all products (of any type) made by manufacturer B.

```
SELECT model, price
FROM Product NATURAL JOIN PC
WHERE maker = B

UNION

SELECT model, price
FROM Laptop NATURAL JOIN Printer
WHERE maker = B
```

- c) Find those manufacturers that sell laptops, but not PC's

```
SELECT maker
FROM Product
WHERE type = 'Laptop'

EXCEPT

SELECT maker
FROM Product
WHERE type = 'PC'
```

3.13 Exercise 6.1.1

If a query has a `SELECT` clause

```
SELECT A B
```

How do we know whether A and B are two different attributes or B is an alias of A

`SELECT A B` is a shortcut for `SELECT A AS B`

3.14 Exercise 6.1.2

Write the following queries, based on our running movie database example.

```
Movies(title, year, length, genre, studioName, producerC#)
StarsIn(movieTitle, movieYear, starName
MovieStar(name, address, gander, birthdate)
MovieExec(name, address, cert#, netword)
Studio(name, address, presC#)
```

This is all done in pgAdmin 4 :).

- a Find the address of MGM studios.

```
SELECT address
FROM Studio
WHERE name = 'MGM';
```

- b Find Sandra Bullock's birthdate.

```
SELECT birthdate
FROM MovieStar
WHERE name = 'Sandra Bullock';
```

- c Find all the stars that appeared either in a movie made in 1980 or a movie with “Love” in the title.

```
SELECT starname
FROM starsin
WHERE LOWER(movietitle) LIKE '%love%' OR movieyear = 1980;
```

- d Find all executives worth at least \$10,000,000.

```
SELECT name
FROM movieexec
WHERE networth >= 10000000;
```

- e Find all the stars who either are male or live in Malibu (have string Malibu as a part of their address).

```
SELECT name
FROM moviestar
WHERE gender = 'm' OR LOWER(address) LIKE '%Malibu%';
```

3.15 Exercise 6.1.5

Let *a* and *b* be integer-valued attributes that may be *NULL* in some tuples. For each of the following conditions (as may appear in a *WHERE* clause), describe exactly the set of (*a*, *b*) tuples that satisfy the condition, including the case where *a* and/or *b* is *NULL*.

- a) *a* = 10 OR *b* = 20
(*a*, *b*) = (10, *NULL*) -> TRUE
(*a*, *b*) = (*NULL*, 20) -> TRUE
- b) *a* = 10 AND *b* = 20
a must be 10 and *b* must be 20
- c) *a* < 10 OR *b* >= 10
a is INTEGER, *b* is INTEGER or *NULL*

3.16 Exercise 6.2.1

Using the database schema of our running movie example FROM Exercise 6.1.1

- a Who were the male stars in *Titanic*?

```
SELECT name
FROM moviestar m JOIN starsin s ON m.name = s.starname
WHERE LOWER(gender) = 'm' AND LOWER(s.movietitle) = 'titanic';
```

- b Which stars appeared in movies produced by MGM in 1995

```
SELECT starname
FROM movies m JOIN starsin s ON m.title = s.movietitle
WHERE m.studio = 'MGM' AND s.movieyear = 1995;
```

- c Who is the president of MGM studios

```
SELECT m.name
FROM movieexec m JOIN studio s ON m.cert = s.presc
WHERE s.name = 'MGM';
```

- d Which movies are longer than *Gone With the Wind*?

```
SELECT M1.title
FROM Movies M1 CROSS JOIN Movies M2
WHERE M1.length > M2.length And LOWER(M2.title) = 'gone with the wind';
```

- e Which executives are worth more than my brother from another mother Merv Griffin

```
SELECT name
FROM movieexec m
WHERE m.networth
>
(SELECT networth
FROM movieexec
WHERE LOWER(name) LIKE '%merv%');
```

3.17 Exercise 6.3.2

White the following queries, based on the database schema

```
Classes(class, type, country, numGuns, bore, displacement)
Ships(name, class, launched)
Battles(name, date)
Outcomes(ship, battle, result)
```

Of Exercise 2.4.3. You should use at least one subquery in each of your answers and write each query in two significantly different ways (e.g., using different sets of operations EXISTS, IN, ALL, ANY)

- a) Find the countries whose ships had the largest number of guns.

```
SELECT country
FROM Classes
WHERE numGuns >= ALL(SELECT numGuns FROM Classes)
```

- c) Find the names of the ships with a 16-inch bore.

```
SELECT name
FROM Ships
WHERE Class IN (SELECT class FROM Classes WHERE bore = 16)
```

3.18 Exercise in lecture

- a) `SELECT MAKER, MIN()`

- b) `SELECT MAKER, COUNT(PC.model)`
`FROM PC NATURAL LEFT OUTER JOIN PRODUCT`
`GROUP BY maker`

3.19 Exercise 6.4.1

Write each of the queries in Exercise 2.4.1 in SQL, making sure that duplicates are eliminated.

- a) What pc models have a speed of at least 3.00?

```
SELECT DISTINCT model
FROM PC
WHERE speed >= 3.00
```

- c) Find the model number and price of all products (of any type) made by manufacturer B

```
SELECT DISTINCT model, price
FROM
  (SELECT maker, model, price FROM product NATURAL JOIN PC)
UNION
  (SELECT maker, model, price FROM product NATURAL JOIN Laptops)
UNION
  (SELECT maker, model, price FROM product NATURAL JOIN Printer)
WHERE maker = 'B'
```

- !!h) Find those manufactures of at least two different computers (PC's or Laptops with speeds of at least 2.80)

```
SELECT maker
FROM
  ((SELECT maker, model, speed FROM PRODUCT NATURAL JOIN PC WHERE speed >= 2.80)
UNION
  (SELECT maker, model, speed FROM PRODUCT NATURAL JOIN Laptop WHERE speed >= 2.80))
GROUP BY maker
HAVING COUNT(maker)>1
```

3.20 Exercise 6.4.2

Write each of the queries in Exercise 2.4.3 in SQL, making sure that duplicates are eliminated.

- !g) Find those classes that had only one ship as a member of that class.

3.21 Exercise 6.4.6

Write the following queries, based on the database schema

- a) Find the average speed of PC's

```
SELECT AVG(speed)
FROM PC
```

- b) Find the average speed of laptops costing over \$1000

```
SELECT AVG(speed)
FROM Laptop
WHERE price > 1000
```

!d) Find the average price of PC's and laptops made by manufacturer "D."

```
SELECT AVG(price)
FROM(
    (SELECT maker, price FROM Product NATURAL JOIN PC)
    UNION ALL
    (SELECT maker, price FROM Product NATURAL JOIN Laptop))
WHERE maker = 'D'
```

e) Find, for each different speed, the average price of a pc

```
SELECT speed, AVG(price)
FROM PC
GROUP BY speed
```

f) Find for each manufacturer, the average screen size of its laptops.

```
SELECT maker, AVG(screen)
FROM Product NATURAL JOIN Laptop
GROUP BY maker
```

g) Find for each manufacturers that make at least tree different models of PC.

```
SELECT maker
FROM Product NATURAL JOIN PC
GROUP BY maker
HAVING COUNT(maker)>2
```

3.22 Exercise 6.4.7

3.23 Exercise 6.5.1

a) Using two INSERT statements, store in the database the fact that PC model 1100 is made by manufacturer C, has speed 3.2, RAM 1024, hard disk 180 and sells for \$2499.

```
INSERT INTO PC(model, speed, ram, hd, price)
VALUES 1100, 3.2, 1024, 180, 2499
```

```
INSERT INTO Product(maker, model, type)
VALUES 'C', 1100, 'PC'
```

b) Insert the facts that for every PC there is a laptop with the same manufacturer, speed, RAM, and a hard disk, a 17-inch screen, a model number 1100 greater and a price 500 more.

```
INSERT INTO Laptop (model, speed, ram, hd, screen, price)
SELECT model + 1100, speed, ram, hd, 17, price + 500
FROM PC
```

```
INSERT INTO Product(maker, model, type)
SELECT maker, model+ 1100, 'Laptop'
WHERE type = 'PC'
FROM Product
```

- c) Insert the facts that for every PC there is a laptop with the same manufacturer, speed, RAM, and a hard disk, a 17-inch screen, a model number 1100 greater and a price 500 more.

```
INSERT INTO Laptop (model, speed, ram, hd, screen, price)
  SELECT model + 1100, speed, ram, hd, 17, price + 500
FROM PC
```

```
INSERT INTO Product(maker, model, type)
  SELECT maker, model+ + 1100, 'Laptop'
WHERE type = 'PC'
FROM Product
```

3.24 Exercise 3.1.1

Consider a relation about people in the United States, including their name, Social Security number, street address, city, state, ZIP code, area code, and phone number (7 digits). What FD's would you expect to hold? What are the keys for the relation? To answer this question, you need to know something about the way these numbers are assigned. For instance, can an area code straddle two states? Can a ZIP code straddle two area codes? Can two people have the same Social Security number? Can they have the same address or phone number

SSN $\rightarrow$ ALL

"street, city, state $\rightarrow$ ZIP code"

3.25 Exercise 3.2.1

Consider a relation with schema $R(A, B, C, D)$ and FD's $AB \rightarrow C$, $C \rightarrow D$, and $D \rightarrow A$.

- a) What are all the nontrivial FD's that follow from the given FD's? You should restrict yourself to FD's with single attributes on the right side.

$(AB)^+ = (ABCD)$,

$AB \rightarrow D$

$C \rightarrow A$

$BD \rightarrow C$

- b) What are all the keys of R?

AB, BC, BD

- c) What are all the superkeys for R that are not keys?

$ABC, ABD, BDC, ABCD$

3.26 Exercise 3.3.3

Repeat Exercise 3.2.1 for the following schemas and sets of FD's:

- i) $S(A, B, C, D)$ with FD's $A \rightarrow B$, $B \rightarrow C$, and $B \rightarrow D$

- a) nontrivial FD's:

$A \rightarrow C$

$A \rightarrow D$

- b) keys of S

A

- c) superkeys of S

$AB, ABC, ABCD$

- ii) $T(A, B, C, D)$ with FD's $AB \rightarrow C$, $BC \rightarrow D$, $CD \rightarrow A$ and $AD \rightarrow b$

- a) nontrivial FD's:
- $$AB \rightarrow D$$
- $$BC \rightarrow A$$
- $$CD \rightarrow B$$
- $$AD \rightarrow C$$
- b) keys of S
- $$AB, BC, CD, AD$$
- c) superkeys of S
- ALL subsets with either AB or BC
- iii) $U(A, B, C, D)$ with FD's $A \rightarrow B, B \rightarrow C, C \rightarrow D$ and $D \rightarrow A$
- a) nontrivial FD's:
- $$A \rightarrow C$$
- $$A \rightarrow D$$
- $$B \rightarrow D$$
- $$B \rightarrow A$$
- $$C \rightarrow A$$
- $$C \rightarrow B$$
- $$D \rightarrow B$$
- $$D \rightarrow C$$
- b) keys of S
- $$A, B, C, D$$
- c) superkeys of S
- All subsets of $ABCD$

3.27 Exercise 3.2.10

Suppose we have relation $R(A, B, C, D, E)$, with some set of FD's, and we wish to project those FD's onto relation $S(A, B, C)$. Give the FD's that hold in S if the FD's for R are:

- a) $AB \rightarrow DE, C \rightarrow E, D \rightarrow C$ and $E \rightarrow A$
- $$\{A\}^+ = \{A\}$$
- $$\{B\}^+ = \{B\}$$
- $$\{AB\}^+ = \{ABCDE\} \text{ in } S, AB \rightarrow C$$
- $$\{BC\}^+ = \{ABCDE\}$$
- $$\{C\}^+ = \{ACE\} \text{ in } S, C \rightarrow A$$
- b) $A \rightarrow D, BD \rightarrow E, AC \rightarrow E$ and $DE \rightarrow B$
- $$\{AC\}^+ = \{ABCDE\} \text{ in } S, AC \rightarrow D$$
- $$\{ABC\}^+ = \{ABDE\} \text{ in } S, AC \rightarrow D$$
- c) $AB \rightarrow D, AC \rightarrow E, BC \rightarrow D, D \rightarrow A$ and $E \rightarrow B$
- $$\{AB\}^+ = \{ABD\}$$
- $$\{BC\}^+ = \{ABCDE\} \text{ in } S, BC \rightarrow A$$
- $$\{AC\}^+ = \{ABCDE\} \text{ in } S, AC \rightarrow D$$
- $$\{ABC\}^+ = \{ABCDE\}$$
- d) $A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow E$ and $E \rightarrow A$
- $$\{A\}^+ = \{ABCDE\} \text{ in } S, A \rightarrow B, A \rightarrow C$$
- $$\{B\}^+ = \{ABCDE\} \text{ in } S, B \rightarrow A, B \rightarrow C$$
- $$\{C\}^+ = \{ABCDE\} \text{ in } S, C \rightarrow A, C \rightarrow B$$

In each case, it is sufficient to give a minimal basis for the full set of FD's of S . (remove duplicates)

3.28 Exercise 3.3.1

For each of the following relation schemas and sets of FD's: do the following:

- i Indicate all the BCNF violations. Do not forget to consider FD's that are not in the given set, but follow from them. However, it is not necessary to give violations that have more than one attribute on the right side.
- ii Decompose the relations, as necessary, into collections of relations that are in BCNF.

a) $R(A, B, C, D)$ with FS's $AB \rightarrow C, C \rightarrow D$ and $D \rightarrow A$.

Firstly, check the closure of the DF's. For this:

$AB \rightarrow ABCD = \text{true}, C \rightarrow ACD = \text{false}$ and $D \rightarrow AD = \text{false}$

So both $C \rightarrow D$ and $D \rightarrow A$ **violate** BCNF

So these should be composed

– $R_1 = C^+ \rightarrow R(ACD)$

This needs decomposing since length is > 2

So decompose again by finding closures of ACD

$C^+ \rightarrow ACD$ This holds true, because of $R_1(ACD)$

$D^+ \rightarrow AD$ This is a **violation**

– $R_{1,1} = D^+ \rightarrow R(AD)$

This holds as the FD has a length of 2

– $R_{1,2} = D + (R - D^+) \rightarrow R(CD)$

This holds as the FD has a length of 2

– $R_2 = C + (R - C^+) \rightarrow R(BC)$

This holds as the FD has a length of 2

The result is $R_{1,1}(AD), R_{1,2}(CD), R_2(BC)$

e) $R(A, B, C, D, E)$ with FS's $AB \rightarrow C, DE \rightarrow C$ and $B \rightarrow D$.

Firstly, check the closure of the DF's. For this:

$AB \rightarrow ABCD = \text{false}, DE \rightarrow CDE = \text{false}$ and $B \rightarrow BD = \text{false}$

So all 3 **violate** BCNF

So these should be composed

– $R_1 = AB^+ \rightarrow R_1(ABCD)$

This does not hold

So decompose again by finding closures of $ABCD$

$AB^+ \rightarrow ABCD$ This holds true due to length 2

$B^+ \rightarrow BD$ This is a **violation**

– $R_{1,1} = B^+ \rightarrow BD$

This holds true due to length 2

– $R_{1,2} = B + (R_1 - B^+) \rightarrow ABC$

$AB^+ \rightarrow ABC$ This is true because $R_{1,2}$

– $R_2 = AB + (R_1 - AB^+) \rightarrow R_2(ABE)$

This does not hold

$AB^+ \rightarrow AB$ This holds true due to length 2

The result is $R_{1,1}(BD), R_{1,2}(ABC), R_2(ABE)$

3.29 Exercise 4.1.1

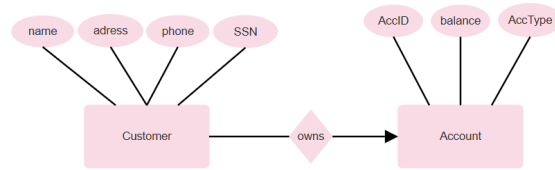


Figure 6: Er-diagram for bank

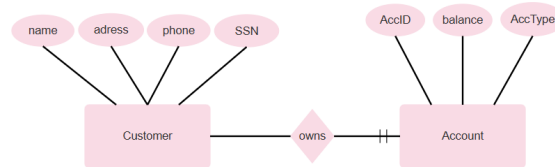


Figure 7: ER-diagram for bank with only one

3.30 Exercise 4.1.2

a)

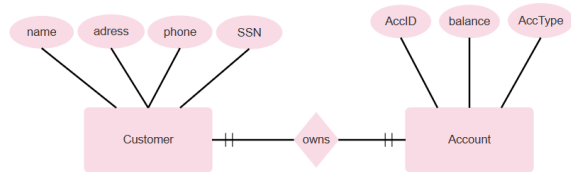


Figure 8: ER-diagram for bank only one both directions

b)

3.31 Exercise 4.1.3

Give an E/R diagram for a database recording information about teams, players and their fans, including:

1. For each team, its name
- 2.
- 3.

3.32 Exercise 4.2.1

In Fig. 4.14 is an E /R diagram for a bank database involving customers and accounts. Since customers may have several accounts, and accounts may be held jointly by several customers, we associate with each customer an “account set,” and accounts are members of one or more account sets. Assuming the meaning of the various relationships and attributes are as expected given their names, criticize the design. What design rules are violated? Why? What modifications would you suggest?

3.33 Exercise 4.3.1

For your E /R diagrams of: Select keys and appropriate referential integrity constraints.

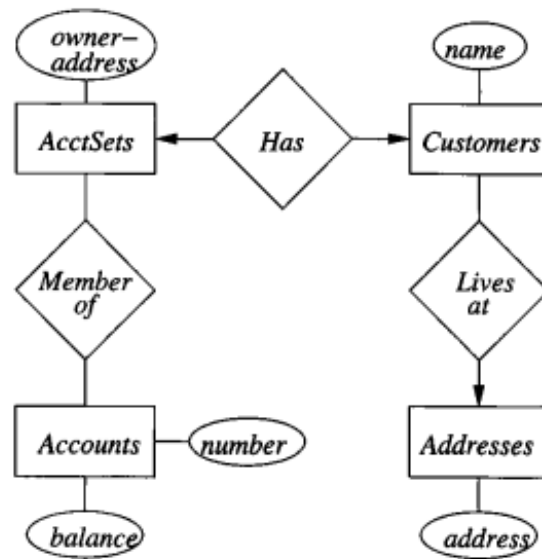


Figure 9: A poor design for a bank database

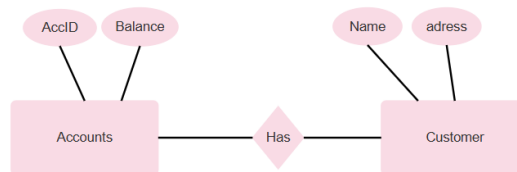


Figure 10: Fixed database

- Exercise 4.1.1.
keys: SSN and numbers
RIC: NONE
- Exercise 4.1.3.
keys: NONE
RIC: Teams, Players and Fans
- Exercise 4.1.6.
keys: NONE
RIC: Person, Mother, Father, Children

3.34 Exercise 4.4.1

One way to represent students and the grades they get in courses is to use entity sets corresponding to students, to courses, and to “enrollments.” Enrollment entities form a “connecting” entity set between students and courses and can be used to represent not only the fact that a student is taking a certain course, but the grade of the student in the course. Draw an E /R diagram for this situation, indicating weak entity sets and the keys for the entity sets. Is the grade part of the key for enrollments?

keys: StudentID, CourseID

RIC: Enrollemnts, StudentOf, CourseOf

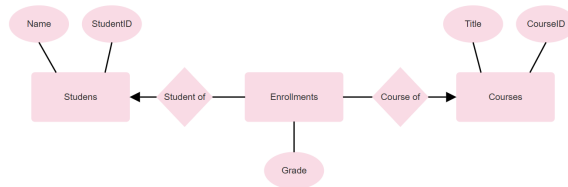


Figure 11: Student in a course E/R

3.35 Exercise 4.5.1

Convert the E /R diagram of Fig. 4.29 to a relational database schema.

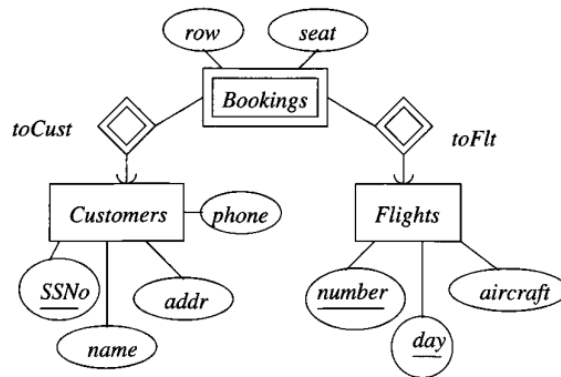


Figure 12: E/R-diagram of a plane-seat booking

- E/R

```

Customer(SSNo,name,addr,phone)
toCust(SSNo)
Bookings(SSNo,number,row,seat)
toFlt(number)
Flights(number,day,aircraft)
  
```

- Object-oriented

```

Booking(row,seat)
BookingC(row,seat,SSNo,name,Addr)
BookingF(row,seat,number,day,aircraft)
BookingCF(row,seat,SSNo,name,Addr,number,day,aircraft)
  
```

-

3.36 Exercise in lecture

```

CREATE TABLE Product
maker
model
type
PRIMARY KEY(model,type)

CREATE TABLE PC
  
```

```

model
speed
ram
hd
price
PRIMARY KEY(model)
FOREIGN KEY(model) REFERENCES Product(maker)

CREATE TABLE Laptop
model
speed
ram
hd
screen
price
PRIMARY KEY(model)
FOREIGN KEY(model) REFERENCES Product(maker)

CREATE TABLE PC
model
color
type
price
PRIMARY KEY(model)
FOREIGN KEY(model) REFERENCES Product(maker)

```

3.37 Exercise 7.1.1

Change pks for databse of section 2.2.8.

- a) The producer of a movie must be someone mentioned in MovieExec. Modifications to MovieExec that violate this constraints are rejected.

```

DELETE FROM movies
WHERE producerc NOT IN (SELECT cert FROM movieexec);

ALTER TABLE movies
  DROP CONSTRAINT producer_fkey,
  ADD CONSTRAINT producer_fkey
    FOREIGN KEY (producerc)
    REFERENCES movieexec(cert);

```

- b) Repeat (a), but violations result in the producerC# in Movie being set to NULL.

```

ALTER TABLE movies
  DROP CONSTRAINT producer_fkey,
  ADD CONSTRAINT producer_fkey
    FOREIGN KEY (producerc)
    REFERENCES movieexec(cert)
    ON DELETE SET NULL
    ON UPDATE SET NULL;

```

- c) Repeat (a), but violations result in the deletion or update of the offending movie tuple.

```
ALTER TABLE movies
  DROP CONSTRAINT producer_fkey,
  ADD CONSTRAINT producer_fkey
    FOREIGN KEY (producerc)
    REFERENCES movieexec(cert)
    ON DELETE CASCADE
    ON UPDATE CASCADE;
```

- d) A movie that appears in StarsIn must also appear in Movie. Handle violations by rejecting the modification.

```
DELETE FROM starsin
WHERE movietitle NOT IN (SELECT title FROM movies);

ALTER TABLE starsin
  ADD CONSTRAINT movienam_fkey
    FOREIGN KEY (movietitle, movieyear)
    REFERENCES movies(title, year);
```

- e) A star appearing in StarsIn must also appear in MovieStar. Handle violations by deleting violating tuples.

```
DELETE FROM starsin
WHERE starnam NOT IN (SELECT name FROM moviestar);

ALTER TABLE starsin
  ADD CONSTRAINT starnam_fkey
    FOREIGN KEY (starnam)
    REFERENCES moviestar(name);
```

3.38 Exercise 7.1.3

Suggest suitable keys and foreign keys for the relations of the PC database:

- Product

```
ALTER TABLE
```

- PC

- Laptop

- Printer

3.39 Exercise in lecture

For ships database

- a. No class may have more than 2 ships

```
CREATE ASSERTION maxTwoShips CHECK(  
  NOT EXISTS(  
    SELECT class  
    FROM ships  
    GROUP BY class  
    HAVING COUNT(name)>=))
```

- b. No class may have less than 2 ships

```
CREATE ASSERTION minTwoShips CHECK(  
  NOT EXISTS(  
    SELECT class  
    FROM classes NATURAL LEFT OUTER JOIN ships  
    GROUP BY class  
    HAVING COUNT(name)<2))
```

3.40 Exercise in lecture

For PCs database

- a. When updating the price of a PC, check that there is no lower priced PC with the same speed.

Using BEFORE

```
CREATE TRIGGER check_pc_price  
BEFORE UPDATE price ON PC  
REFERENCING OLD ROW AS OLD, NEW ROW AS NEW  
FOR EACH ROW  
WHEN (EXISTS (SELECT model FROM PC  
              WHERE speed = NEW speed  
                price < NEW price)  
UPDATE PC  
SET price = OLD price  
WHERE model = OLD model
```

- b. No class may have less than 2 ships

Using AFTER

```
CREATE ASSERTION minTwoShips CHECK(  
  NOT EXISTS(  
    SELECT class  
    FROM classes NATURAL LEFT OUTER JOIN ships  
    GROUP BY class  
    HAVING COUNT(name)<2))
```



```

1) CREATE TRIGGER AvgNetWorthTrigger
2) AFTER UPDATE OF netWorth ON MovieExec
3) REFERENCING
4)     OLD TABLE AS OldStuff,
5)     NEW TABLE AS NewStuff
6) FOR EACH STATEMENT
7) WHEN (500000 > (SELECT AVG(netWorth) FROM MovieExec))
8) BEGIN
9)     DELETE FROM MovieExec
10)    WHERE (name, address, cert#, netWorth) IN NewStuff;
11)    INSERT INTO MovieExec
12)        (SELECT * FROM OldStuff);
13) END;

```

Figure 13: Constraining the average net worth

3.41 Exercise 7.5.1

Write the triggers analogous to figure 7.6 for the insertion and deletion events on MovieExec

```

CREATE OR REPLACE TRIGGER AvgNetWorthTrigger
AFTER UPDATE OR INSERT OR DELETE ON movieexec
FOR EACH ROW
EXECUTE PROCEDURE updateExec();

CREATE OR REPLACE FUNCTION updateExec() RETURNS TRIGGER AS
$$BEGIN
IF 500000 > (SELECT AVG(networth) FROM movieexec) THEN
    DELETE FROM movieexec
    WHERE (OLD.cert) = (NEW.cert);
    INSERT INTO movieexec
        (SELECT * FROM NEW);
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

3.42 Exercise 6.6.1

This and the next exercises involve certain programs that operate on the two relations. Product, PC from our running PC exercise. Sketch the following programs, including SQL statements and work done in a conventional language. Do not forget to issue **BEGIN TRANSACTION**, **COMMIT**, and **ROLLBACK** statements at the proper times and to tell the system your transactions are read-only if they are.

- a) Given a speed and amount of RAM (as arguments of the function), look up the PC's with that speed and RAM, printing the model number and price of each.

```
SET TRANSACTION READ ONLY
BEGIN TRANSACTION
    EXECUTE FUNCTION findPC(speed, ram)
COMMIT;

CREATE OR REPLACE FUNCTION findPC(fspeed DECIMAL, fram DECIMAL) RETURNS SETOF PC AS
$$BEGIN
    RETURN QUERY
    SELECT model, price
    FROM PC AS P
    WHERE (P.speed = speed AND P.ram = ram)
END IF;
END;
$$
LANGUAGE plpgsql;
```

- b) Given a model number, delete the tuple for that model from both PC and Product.

```
SET TRANSACTION READ ONLY
BEGIN TRANSACTION
    EXECUTE FUNCTION deletePC(model)
COMMIT;

CREATE OR REPLACE FUNCTION findPC(fmodel INT) RETURNS VOID AS $$
BEGIN
    DELETE FROM Product
    WHERE fmodel = Product.model
    DELETE FROM PC
    WHERE fmodel = PC.model
END;
$$ LANGUAGE plpgsql;
```

- c) Given a model number, decrease the price of that model PC by \$100.

```
SET TRANSACTION READ ONLY
BEGIN TRANSACTION
    EXECUTE FUNCTION decreasePricePC(model)
COMMIT;

CREATE OR REPLACE FUNCTION findPC(fmodel INT) RETURNS VOID AS $$
BEGIN
    UPDATE PC
    SET price = price - 100
    WHERE model = fmodel;
END;
$$ LANGUAGE plpgsql;
```

- d) Given a maker, model number, processor speed, RAM size, hard-disk size, and price, check that there is no product with that model. If there is such a model, print an error message for the user. If no such model existed in the database, enter the information about that model into the PC and Product tables.

```
SET TRANSACTION READ ONLY
BEGIN TRANSACTION
    EXECUTE FUNCTION decreasePricePC(model)
COMMIT;

CREATE OR REPLACE FUNCTION findPC(fmaker CHAR(20), fmodel INT, fspeed DECIMAL,
                                   fram DECIMAL, fhd DECIMAL, fprice DECIMAL)
    RETURNS VOID AS $$
BEGIN
    IF EXISTS (SELECT * FROM Product WHERE fmodel = Product.model)
        AND EXISTS (SELECT * FROM PC WHERE fmodel = PC.model)
        RAISE NOTICE ("ERROR: model already exists in database")
    ELSE
        INSERT INTO Product(maker, model, type)
        VALUES(fmaker, fmodel, 'PC')
        INSERT INTO Product(model, speed, ram, hd, price)
        VALUES(fmodel, fspeed, fram, fhd, fprice)
    END;
$$ LANGUAGE plpgsql;
```

3.43 Exercise 9.4.1

write PSM procedures or functions to perform the following tasks:

- a Given the name of a movie studio, produce the net worth of its president.

```
CREATE OR REPLACE FUNCTION findNetWorthOfPresident(IN sn CHAR(20)) RETURNS INT AS
$$
BEGIN
    pres := (SELECT presC FROM studio WHERE name = sn);
    return (SELECT networth FROM movieexec WHERE pres = cert);
END;
$$
LANGUAGE plpgsql;
```

- b Given a name and address, return 1 if the person is a movie star but not an executive, 2 if the person is an executive but not a star, 3 if both, and 4 if neither.

```
CREATE FUNCTION b(star_name character varying(30), star_address character varying(30)) RETURN
$$
BEGIN
    IF EXISTS (SELECT name FROM moviestar WHERE name = star_name AND address = star_address) AND NO
        RETURN 1;
    ELSIF NOT EXISTS (SELECT name FROM moviestar WHERE name = star_name AND address = star_address)
        RETURN 2;
    ELSIF EXISTS (SELECT name FROM moviestar WHERE name = star_name AND address = star_address) AND
        RETURN 3;
    ELSE
        RETURN 4
    END IF;
END
$$
LANGUAGE plpgsql;
```

3.44 Exercise 9.3.1

Write the following embedded SQL queries, based on the database schema for Products

- a Ask the user for a price and find the PC whose price is closest to the desired price. Print the maker, model number, and speed of the PC.
- b Ask the user for minimum values of the speed, RAM, hard-disk size, and screen size that they will accept. Find all the laptops that satisfy these requirements. Print their specifications (all attributes of Laptop) and their manufacturer.

3.45 Exercise 13.2.1

The Megatron 777 disk has the following characteristics:

1. There are ten surfaces, with 100000 tracks each.
2. Tracks hold an average of 1000 sectors of 1024 bytes each.
3. 20% of each track is used for gaps.
4. The disk rotates at 10000 rpm.
5. The time it takes the head to move n tracks is $1 + 0.0002n$ milliseconds.

Answer the following questions about the Megatron 777.

- a) What is the capacity of the disk?

$$10 \text{ surfaces} \cdot 100000 \text{ tracks} \cdot 1000 \text{ sectors} \cdot 1024 \text{ bytes} = \underline{1TB}$$

- b) If tracks are located on the outer inch of a 3.5-inch-diameter surface, what is the average density of bits in the sectors of a track?

We know that $C = 2\pi r$

$$\text{so } 2\pi \cdot \frac{3.5 \text{ inches}}{2} = 10.9956 \text{ inches}$$

since there are a 20% gap $1000 \cdot 1024 \cdot 8 \cdot 0.8$ bits/bytes on a track

$$\text{Density must be } \frac{\text{bits}_{\text{track}}}{\text{circ}} = \underline{596022 \text{ bits/inch}}$$

- c) What is the maximum seek time?

$$\text{we use 5th info for } t_{\text{seek}} = 1 + 0.0002 \cdot 100000 = \underline{21 \text{ ms}}$$

- d) What is the maximum rotational latency?

$$\text{since disk rotates at } 10000 \text{ rpm, 1 rotation must be } 1/10000 = \underline{6 \text{ ms}}$$

3.46 Exercise 13.3.1

Suppose we are scheduling I/O requests for a Megatron 747 disk, and the requests in Fig. 13.9 are made, with the head initially at track 32,000. At what time is each request serviced fully if:

Cylinder of Request	First time available
8000	0
48000	1
4000	10
40000	20

Figure 13.9: Arrival times for four block-access requests

Figure 14: Arival times for exercise

For this: $t_{seek} = 1 + (\frac{tracks\#}{4000})$, $rot_{lat} = 4.17$ and $t_{transfer} = 0.13$

- a) We use the elevator algorithm (it is permissible to start moving in either direction at first).

Cylinder	t_r	t_{hit}	t_{rest}
8000	0	7	11.3
4000	10	13.3	17.6
48000	1	28.6	33.9
40000	20	35.9	41.2

Table 1: The elevator algorithm

- b) We use first-come-first-served scheduling.

Cylinder	t_r	t_{hit}	t_{rest}
8000	0	7	11.3
48000	1	22.3	26.6
4000	10	37.6	42.9
40000	20	52.9	57.2

Table 2: First-come-first-served scheduling

3.47 Exercise 13.4.1

Compute the parity bit for the following bit sequences:

- a) 00111011

1's are odd so 1

- b) 00000000

1's are even so 0

- c) 10101101

1's are odd so 1

3.48 Exercise 13.4.2

We can have two parity bits associated with a string if we follow the string by one bit that is a parity bit for the odd positions and a second that is the parity bit for the even positions. For each of the strings in Exercise 13.4.1, find the two bits that serve in this way

00111011Even $0 + 1 + 1 + 1 = 3$, so 1
 Odd $0 + 1 + 0 + 1 = 2$, so 0
 00000000Even $0 + 0 + 0 + 0 = 0$, so 0
 Odd $0 + 0 + 0 + 0 = 0$, so 0
 10101101Even $1 + 1 + 1 + 0 = 3$, so 1
 Odd $0 + 0 + 1 + 0 = 1$, so 0

3.49 Exercise 13.4.6

Suppose we are using a RAID level 4 scheme with four data disks and one redundant disk. As in Example 13.9 assume blocks are a single byte. Give the block of the redundant disk if the corresponding blocks of the data disks are:

a) 01010110, 11000000, 00111011, 11111011

01010110	
11000000	
00111011	
11111011	
01010110	

Table 3: RAID 4 scheme

b) 11110000, 11111000, 00111111, 00000001

11110000	
11111000	
00111111	
00000001	
00110110	

Table 4: RAID 4 scheme

3.50 Exercise 13.7.1

A patient record consists of the following fixed-length fields: the patient's date of birth, social-security number, and patient ID, each 10 bytes long. It also has the following variable-length fields: name, address, and patient history. If pointers within a record require 4 bytes, and the record length is a 4-byte integer, how many bytes, exclusive of the space needed for the variable length fields, are needed for the record? You may assume that no alignment of fields is required.

3.51 Exercise 14.1.1

Suppose blocks hold either three records, or ten key-pointer pairs. As a function of n , the number of records, how many blocks do we need to hold a data file and:

- (a) A dense index Since 1 block holds at most 3 records or 10 key pointer pairs $\lceil \frac{n}{3} \rceil + \lceil \frac{n}{10} \rceil$
 (b) A sparse index? The same as (a) but one pointer may point at a block of 3 records $\lceil \frac{n}{3} \rceil + \lceil \frac{n}{3 \cdot 10} \rceil$

3.52 Exercise 14.2.1

Suppose that blocks can hold either ten records or 99 keys and 100 pointers. Also assume that the average B-tree node is 70% full; i.e., it will have 69 keys and 70 pointers. We can use B-trees as part of several different structures. For each structure described below, determine (i) the total number of blocks needed for a 1,000,000-record file, and (ii) the average number of disk I/O's to retrieve a record given its search key. You may assume nothing is in memory initially, and the search key is the primary key for the records.

a) The data file is a sequential file, sorted on the search key, with 10 records per block. The B-tree is a dense index

i Since 1 block holds at most 10 records or 70 key pointers

For leaf layer: $\frac{1000000}{10} = 100000$

For first layer above leafs: $\lceil \frac{100000}{70} \rceil = 14286$

For second layer above leafs: $\lceil \frac{14286}{70} \rceil = 205$

For third layer above leafs: $\lceil \frac{205}{70} \rceil = 3$

For fourth layer above leafs: $\lceil \frac{3}{70} \rceil = 1$ (root)

$100000 + 14286 + 205 + 3 + 1 = 114495$ blocks

ii for this: $\lceil \log_{70}(1000000) \rceil$

3.53 Exercise 14.1.1